n = 83: a faster root-index build for the closure + index-lift row (Tier 1 design)

2026-10-20

n = 83: a faster root-index build for the closure + index-lift row (Tier 1 design)

Status (2026-10-20): design frozen, run launched; no result yet. Nothing below is a measured speedup. The profile and the candidate savings come from the shared cloud-agent VM. They are **local diagnostics**, used only to rank the candidates and size the design. The verdict will come from the frozen Tier 1 run (see Trigger).

Why this run

The n = 83 a = 1 baseline row (`docs/ic/BASELINES.md`, `baselines.json`) is the closure + index-lift M38I producer:

- the experiment is `koblitz-n83-closure-indexlift-20261019` (PR #997);
- the derived producer's SHA-256 is `1d328b2e...`;
- its Tier 1 qualification is `koblitz-n83-closure-indexlift-rep-20261020` (PR #1002, run 39359922687), which measured R = 7.4133 [7.1999, 7.6362] over 30 alternating pairs.

In that run the row spends its 21.74 s end-to-end cold time as follows:

Stage	Time (s)	Share
root-index build	15.63	72%
full scan-order self-test	1.70	
rank stage	3.74	17%
online	0.35	

The index build is now the dominant cost.

This experiment profiles that build, ranks the candidate speedups, and implements the mechanical ones. Each implemented change leaves the output byte for byte the same: the same stored states, table slots, relations, probes and self-test verdict. A Tier 1 protocol with repeated alternating pairs then measures the new producer against the row.

1. Profile of the index build (local diagnostic)

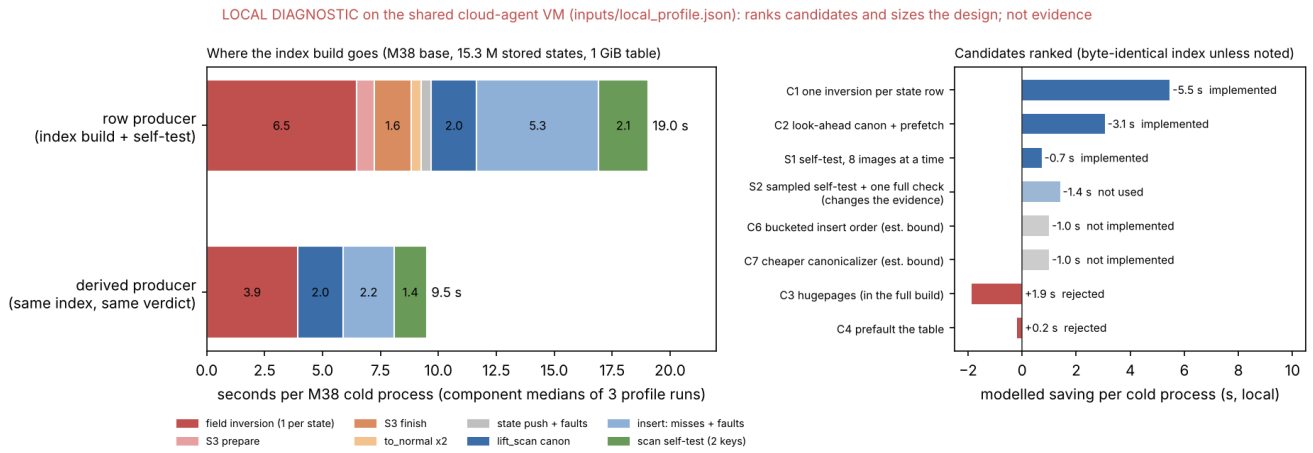


Figure 1: LOCAL DIAGNOSTIC: component medians of three profile processes on the M38 closure header, and the candidates ranked by modelled saving. Rendered from `inputs/local_profile.json` by `make_figures.py` (svg, pdf).

Method. `profile/profile_patch.py` patches a *scratch copy* of the row's producer. That copy is never committed as a producer. It adds `KIC_INDEX_PROFILE` instrumentation (`Instant` timers and `getrusage` fault counters), which does the following:

- builds the parent's index in place;
- times each component alone (S3 prepare, field inversion, finish, `to_normal`, push, `lift_scan`, insert);
- times each candidate variant;
- asserts that every variant reproduces the parent's states and slot words.

`perf` is not installed on the VM (`perf_event_paranoid` 2), so the profile uses these counters instead.

There were three processes on the M38 header (608 orbits, 15,341,360 stored states, 30,682,720 roots, 30,366,551 entries, 2^26 slots of 16 B = 1 GiB), each pinned with `taskset -c 2`, built with rustc 1.97 (the workflow pins 1.99). `profile/summarize.py` writes `inputs/local_profile.json`.

Component of the row's index build (M38)	Median (ms)	What it is
state loop	**9,389**	15.3 M `S3Solver128::roots` calls and pushes
field inversion	6,464	one Itoh-Tsujii inversion per state (about 420 ns each)
S3 prepare	780	squares and products before the inversion
S3 finish	1,579	half-trace, products, trace test
`to_normal` of both roots	437	2 x 11 byte-table lookups
push and first touch	434	736 MB of states, 179,782 minor faults (`Vec` grows by doubling)
table insert loop	**7,223**	30.7 M `index_canonical128` + `insert_if_absent`
`lift_scan` canonicalization	1,961	about 64 ns per root
insert: cache and TLB misses, first touch	5,263	random slots in a 1 GiB table; 524,288 minor faults (2 per 4 KiB page: a read maps the zero page, then the write copies it)

Component of the row's index build (M38)	Median (ms)	What it is
table allocation	0.02	`calloc`, lazily zeroed
index build total	**16,921**	`timing_ms.index_build`
scan self-test, 2 keys (outside `index_build`, inside setup)	2,125	2 x 2^26 Feistel evaluations plus a 3.8 MB bitmap
inverse self-test, 2 keys	2	65,536 samples each

There is no I/O in the build: the base header is read before it, in `base_load`, about 0.37 s.

The index digests are computed after the target, outside every timed interval. Index memory layout: `State128` is 48 B (two `u128` roots plus three `u16`), and the compact table has one `u128` word per slot.

2. Candidates, ranked by modelled saving

Every saving below is the local median per M38 cold process. Unless noted, the output is byte-identical.

#	Candidate	Modelled saving	Output	Status
C1	One field inversion per `(left, right)` row of states (`RootBatch128`, Montgomery's trick), with the states vector allocated once	**5.5 s** (9.39 to 3.93)	identical: the inverse is unique	**implemented**
C2	Canonicalize root i+32 ahead and prefetch its home slot while inserting root i	**3.0 s** (insert 5.26 to 2.20)	identical: same inserts, same order	**implemented**
S1	The same exhaustive self-test, computing 8 Feistel images before their 8 checks	**0.7 s** (2.06 to 1.40)	identical verdict, same domain	**implemented**
C1'	Batches of 4 or 16 rows instead of 1	0.0 to 0.3 s more	identical	not taken (within noise)
C6	Bucketed or radix-ordered insert (sort the inserts by home slot)	at most about 1 s (the remaining insert minus faults)	linear probing with first-wins is order-dependent; clusters that cross a bucket boundary need an extra proof	not implemented
C7	A cheaper canonicalizer than `lift_scan`	at most about 1 s (1.96 s total)	needs a new exhaustive equivalence proof	not implemented
C3	`MADV_HUGEPAGE` on the table	1.0 s in the insert-only microbenchmark, but **1.9 s slower** in the full build (2 MiB zeroing and compaction under `defrag=madvise`)	identical	rejected (host-dependent, slower)

#	Candidate	Modelled saving	Output	Status
C4	Prefault the table by sequential writes	-0.2 s (the 0.46 s prefault equals the double faults it saves)	identical	rejected
S2	**Evidence-changing:** run the full self-test once per binary and base before timing, and only a sampled check per process	1.4 s more (all of S1's remaining cost)	**changes what each process proves**	**listed separately, not used**

Implemented together (C1 + C2 + S1), these cut the local index build from 16.4 s to 8.2 s and the self-test from 2.06 s to 1.40 s. The local cold process went from 19.1 s to 10.2 s, both on point s000 of the template panel (`profile/raw/cold_*_s000.stdout.txt`).

Applied to the template run's means, the model predicts the following. These are estimates for sizing the design, not results:

- index build about **2.0x**;
- cold about **1.87x**;
- **end to end about 1.63x** (21.74 s to about 13.36 s).

What the self-test protects, and why S2 is not used. The self-test checks that the Feistel scan order (`ScanPermutation`) visits every logical state index exactly once, for the process's own state count and two fixed keys. Its verdict is a deterministic function of the code and of `len`, so repeating it per process adds no new information about the data. What it does give is per-process, in-receipt evidence that the code that ran had a bijective scan order on the state count it scanned. The crate tests cover small lengths and the K600 count, and the sampled inverse check covers random points.

S2 would move the exhaustive check to one pre-timing process per binary and base, and leave the timed processes with the sampled check. Both checks target the same predicate. But S2 changes the evidence: a timed receipt would no longer show the exhaustive check for itself, and the template's arms, gates and planted control all assume `scan_selftest=full`. So S2 is listed here and not used. S1 keeps the exhaustive check in every process.

3. What is implemented: a counted derivation

`make_indexbuild_source.py` follows the same pattern as the parents' `make_*_source.py` files:

1. It re-derives the row's producer with the parent's `make_online_source.py` (verbatim), which checks rank-probe `d4394df3...`, index-lift `7df72603...`, closure `8916e034...` and the composition.
2. It requires that output to be `1d328b2e...`.
3. It applies **7 counted replacements**, each of which must occur exactly once:
   - the header comment;
   - one constant;
   - the state loop (C1);
   - the compact insert loop (C2);
   - the self-test (S1);
   - one summary field (`index_build_pipeline`);
   - one appended test module.
4. `check_inverse()` undoes the replacements on every derivation and must get the parent's bytes back.

The derived source is `bc6a31c7...` (`koblitz_orbit_dlp_fast_online_indexbuild.rs`), and the binary is `ic_online_indexbuild_r770`. Both IC binaries are built by one `cargo build` from the parent's lock file. `online/Cargo.toml` has one added `[[bin]]`.

`restore.py` checks the following on every build, and the workflow runs it before building:

- the parent protocol `ff5b4cbf...`;

- side A's producer equals the parent's frozen `1d328b2e...`;
- both closure headers;
- the 23 files copied from the parent (`VERBATIM`);
- the 2 files copied from the template (`TEMPLATE`: `run_schedule.py` and `check_panel_disjoint.py`, the same bytes as on main after #1002);
- every local pin in `protocol.json`.

`arm.py` is the parent's with two marked gates added:

- `index_build_pipeline` must equal the arm's value (none on side A);
- the planted `lift_plant` arm must check exactly 2 x stored states roots, and every other arm 0.

Exactness argument (also in `protocol.json`, `exactness_argument.indexbuild`):

1. *State loop.* `roots` computes prepare, one inversion of the denominator `a*ps` (a zero denominator takes the scalar fallback) and finish. `RootBatch128::solve` runs the same prepare. It then inverts the product of the row's non-zero denominators once and recovers each inverse as (prefix product) x (inverse of the running product). That is the denominator's field inverse, because inverses in GF(2^83) are unique. Finish then returns the same roots in the same order. The states are pushed in the parent's order.
2. *Insert loop.* `index_canonical128` is a pure function of the root (plus the planted counter, which is called once per root, in order). Computing it 32 roots early changes no key or shift. The prefetch is a hint without architectural effect. `insert_if_absent` receives the same `(key, value)` sequence, so every slot word is the parent's.
3. *Self-test.* The parent returns false iff some image of `[0, domain)` is at least `domain` or an in-range image repeats, and otherwise returns `visited == len`. The lanes loop evaluates the same `apply` on the same indices and the same checks in index order, so it returns the same boolean.

4. Exactness checks before freezing (cloud-agent VM, correctness only)

- **Crate tests** (`indexbuild_tests`, 4 tests, in the online crate's `cargo test --bins`, which the workflow runs before timing). The parent's state loop, insert loop and self-test are kept there verbatim as references. The tests check:
  - widths 1, 2, 9 and 17;
  - every index canonicalizer (selected, lift, lift_plant), folded and planted fold modes, compact and base tables;
  - that the states, slot words, entries and planted-check counts equal the parent's;
  - that the self-test verdict equals the parent's for every length 1..1500 with 5 keys, for planted non-bijections (a collision, an out-of-domain image, a dropped image, shorter vectors), and for the frozen M38/M10 (30,682,112) and K600 (29,880,000) counts with both keys.

  All 25 tests of the derived binary pass, including the parent's 21.
- **Per-base output equality** (`inputs/local_exactness.json`, from `profile/raw/exactness/` by `profile/summarize_exactness.py`). Run through the wrapper with the full gate set, the derived producer:
  - reproduced the frozen **layout digests** (`stored_states_blake3`, `table_slots_blake3`), logical-state digests and table-map digests on **K600** (`d8cfe443.../5f53764e...`), **M38** (`f7a8d6b8.../6c08c6c2...`) and **M10** (`c90b501b.../4973cfe5...`);
  - reproduced the **Q0 identities** on all three bases (1,987,852, 1,612,533 and 2,130,165 probes, the frozen relations, scalar 3141592653589793);
  - wrote rank tables equal to the frozen verified tables on M38 (seed 1201) and M10 (seed 301);
  - on M38 seed 1201, gave the **same rank relation digest, per-attempt probe vector, root calls, frame and point-addition counters** as the row's producer.
- **Smoke of the frozen schedule** (O1 and P1, after freezing; see `smoke/README.md`).

5. Frozen design (Tier 1, repeated alternating pairs)

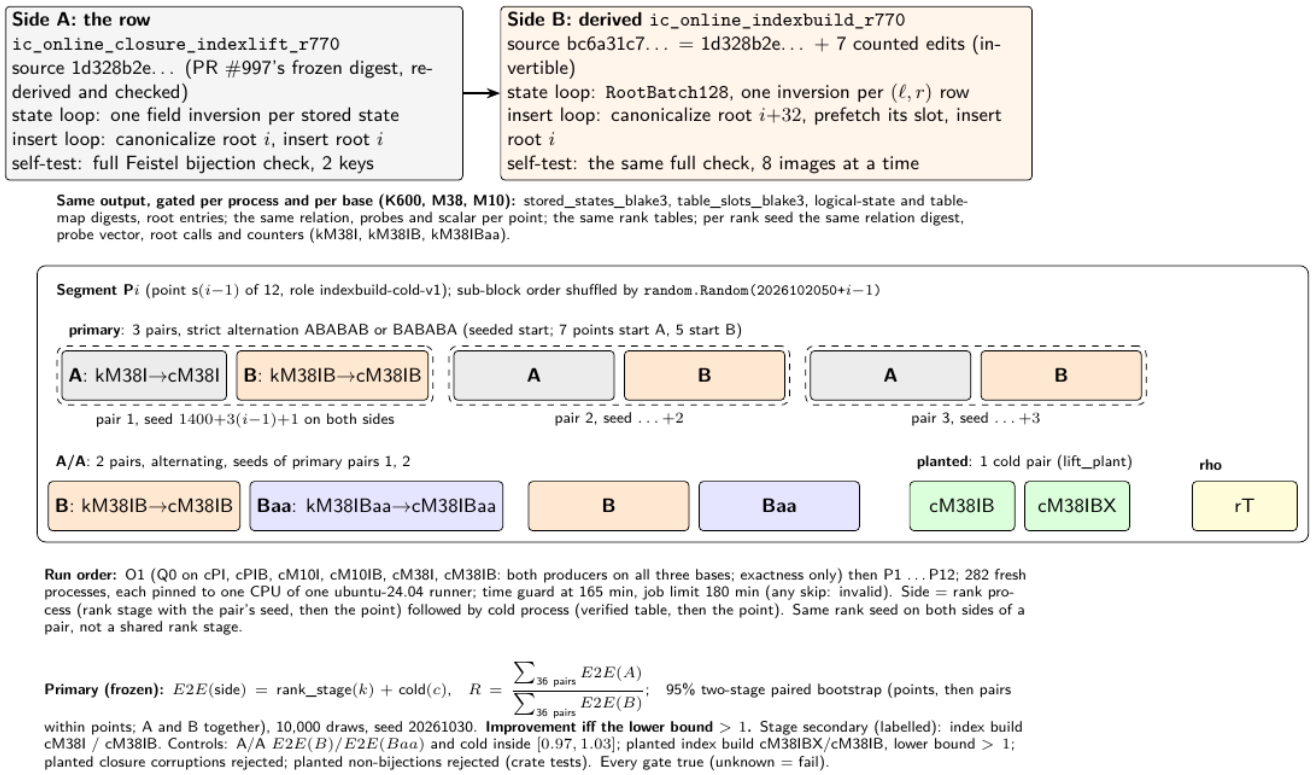


Figure 2: Design (from the frozen protocol and schedule; not a result). Source figures/design.tex, vector figures/design.pdf.

The design follows the Tier 1 template (`koblitz-n83-closure-indexlift-rep-20261020`): one GitHub `ubuntu-24.04` runner, and a fresh process for every run, pinned to one CPU with a clean environment. It keeps the template's:

- `run_schedule.py` (verbatim): CPU record, placement, scheduling counters, cgroup, steal and fault counters, time guard;
- two-stage paired bootstrap;
- A/A band;
- fail-closed verifier, extended with two gates.

Sides and pairs.

side	processes, in order	binary
A	kM38I (rank stage, 16 attempts on M38, pair's seed, then the point), then cM38I (verified table, the point)	the row, `ic_online_closure_indexlift_r770`
B	kM38IB (same seed), then cM38IB	derived, `ic_online_indexbuild_r770`
Baa	kM38IBaa, then cM38IBaa	A/A copy of B

- **Primary:** 12 points x **3 pairs = 36 primary pairs**. Each point's six sides strictly alternate, ABABAB or BABABA; 7 points start with A and 5 with B. Seed of pair j at point i: $1400 + 3i + j$.
- **A/A:** 2 pairs per point (B against Baa), with the seeds of primary pairs 1 and 2.

- **Planted:** 1 cold pair per point, cM38IB against cM38IBX. In cM38IBX, `KIC_INDEX_CANON=lift_plant` also runs the selected canonicalizer on every root and asserts that it equals `lift_scan`. That is slower (locally 1.8-2.2x on the index build), and the layout is the same.
- **rho:** one rT per point, seed 1030000 + point seed.
- **O1:** Q0 on cPI, cPIB, cM10I, cM10IB, cM38I and cM38IB, so both binaries run on all three bases. This is exactness only.
- In total, **282 processes**. Sub-block order and starts come from `random.Random(2026102050+i)`.

Rank stage: the same seed per pair, not a shared rank stage. Each side runs its own rank process on its own binary. The change touches only how the index is computed, so the rank stage's work is identical. This is gated per seed across kM38I, kM38IB and kM38IBaa: identical relation digests, probe vectors, root calls and counters. Timing the rank stage on both sides keeps any code-layout effect of the new binary on the rank loop inside E2E, rather than assuming it is zero. A shared-rank design would have to assume that. This costs about 15 s of process time per pair.

Primary (pre-registered).

- E2E(side) = `timing_ms.rank_stage` (rank process) + `cold_in_process_ms` (cold process).
- R = sum E2E(A) / sum E2E(B) over the 36 pairs.
- 95% two-stage paired bootstrap: points, then pairs within points, with A and B of a pair kept together; 10,000 draws, `random.Random(20261030)`.
- **Improvement iff the lower bound > 1.** Regression iff the upper bound < 1. Otherwise not separated.
- **Invalid** if any gate is false or unknown, if any control fails, or if any row is missing (including time-guard skips).

Stage secondary (labelled): the index-build ratio, cM38I / cM38IB `timing_ms.index_build`, with the same bootstrap. The run also reports the index build in the rank processes.

Other secondaries (descriptive):

- geometric mean of the pair ratios;
- cold, setup, scan self-test, online, rank stage (expected about 1), CPU time, wall time, peak RSS and cold minor faults;
- per point, by pair position and by side order;
- probe law of kM38I and kM38IB;
- S and the gap to rT.

Controls.

- **A/A:** E2E(B)/E2E(Baa) and cold cM38IB/cM38IBaa, each inside [0.97, 1.03] over 24 pairs.
- **Planted timing:** index build cM38IBX/cM38IB, lower 95% bound > 1 over 12 pairs.
- **Planted exactness:**
  - the three closure corruptions rejected before timing (`check_log_table.py`);
  - the planted non-bijections rejected in the crate tests;
  - every cM38IBX checks 2 x 15,341,360 roots and keeps the frozen layout.

Gates. All gates are fail closed; an unknown gate counts as a failure. They are the template's gates plus the following:

- every row ran **its arm's binary**, by name and by the digest that `runner_start.json` lists;
- one index and layout digest set per base on **K600, M38 and M10**, across both binaries, equal to the frozen one;
- Q0 identities on all six O1 arms;
- the probe-law band for kM38I over 36 seeds (576 attempts).

Panel and seeds. The panel is 12 new public points, role `indexbuild-cold-v1`, seeds 0..11 (`make_targets.py`). They are checked against every role of the law named in `EARLIER_ROLES`, including the template's `closureindexlift-rep-cold-v1`.

A role-agnostic x/y scan (`inputs/panel_disjointness.json`) found **0 hits** in all of these:

- origin/main at `5332b222`;
- 853 other origin branches (1,632,523 changed files);

- the scratch worktrees `n83-blrp`, `n83-cli`, `n83-cli-rep`, `n83-closure`, `n83-indexlift`, `n83-k899`, `n83-knownlog`, `n83-onlineframe` and `n83-rotkey`.

A `git grep` of the new main (`9de0ed9f0`, after #1002 merged) also finds none. As a positive control, the same grep finds a template panel coordinate in the template's `cold_targets.json`, `schedule.json` and `runs.jsonl`.

The rank seeds 1401-1436 are new (`inputs/seed_disjointness.json`). That scan searched every n83 schedule and protocol on origin/main, on 83 n83 branches and in the worktrees. Its positive control finds the template's 1201-1230.

6. Power and runtime

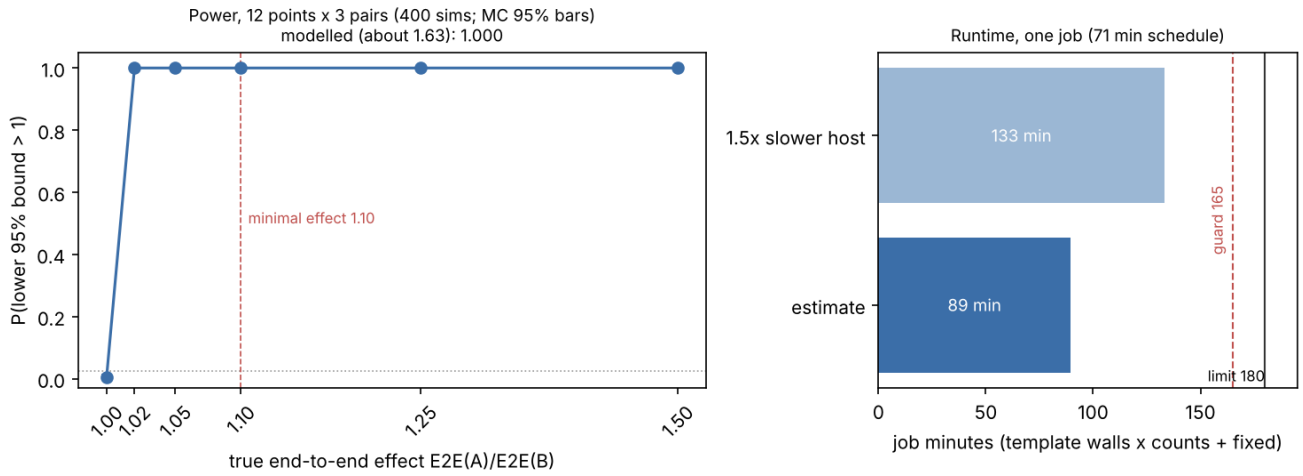


Figure 3: Design-stage power and runtime (not a result), from `analysis/power.json` by `make_figures.py` (svg, pdf).

The prior is the template's archived run 39359922687 (`make_power_prior.py`, pinned `runs.jsonl` digest):

- side A's 30 primary pairs (kM38I rank stage; cM38I cold, index build and self-test);
- its 20 A/A ratios, used as per-process noise factors. This is conservative by about sqrt(2).

Side B is modelled as A minus (1 - f) x index build and self-test. f comes from the local profile (0.499 and 0.681) in the `modelled` cell, or is set to hit each effect cell. There are 400 simulations per cell and 500 draws each.

true end-to-end effect	P(lower bound > 1)
1.00 (no gain; type I)	0.005
1.02	1.00
1.05	1.00
1.10 (minimal effect of interest)	**1.00**
1.25	1.00
modelled, about 1.63	**1.00** (median R 1.627, median interval hi/lo 1.028)

The controls behave as follows in simulation:

- the A/A point estimates are inside the band in 100% of simulations;
- the probe-law gate (576 attempts) passes at the archived law in 100% and rejects a law 1.5x worse in 100%;
- the planted lower bound exceeds 1 in 100%, modelled conservatively at 1.5x.

Runtime. The process walls are the template's, with side B's reduced by the modelled saving, giving:

- about 350 s per point segment;
- a **71.5 min** schedule;

- a **job of about 89 min** with 16 min fixed before (restore, build of three binaries, both crates' tests, table checks) and 2 min after;
- **133 min** on a host 1.5x slower.

The time guard is **165 min** and the job limit is **180 min**, so one job fits within the 3 h budget. Any time-guard skip makes the run invalid.

Trigger

The run is the workflow `.github/workflows/n83-indexbuild.yml` (`workflow_dispatch`, plus a push trigger restricted to branch `claude/n83-indexbuild` and to this experiment's non-receipt paths). The trigger outcome and the run URL are recorded in Run.

Run

(Filled in when the run is launched; the result is added with the archived receipt.)

Files

File	Role
`make_indexbuild_source.py`	the derivation of side B's producer (7 counted, invertible edits on `1d328b2e...`)
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol (sha256 `3073d0c5...`), its builder, frozen run order (`5b1db827...`)
`restore.py`	restore crypto 770f6a69, derive both producers, regenerate the closure headers, check every parent, template and local pin
`arm.py`	the parent's wrapper with two marked gates
`run_schedule.py`, `check_panel_disjoint.py`	the template's, verbatim
`verify_receipt.py`, `test_verify_receipt.py`	gates, statistics, fail-closed decision rule (29 unit tests)
`make_targets.py`, `inputs/cold_targets.json`, `inputs/panel_disjointness.json`, `check_seed_disjoint.py`, `inputs/seed_disjointness.json`	the new panel, seeds, and their disjointness records
`profile/`	profiling patch (scratch copies only), raw profile and exactness records, summarizers; `inputs/local_profile.json`, `inputs/local_exactness.json`
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39359922687), simulation and result
`make_*_source.py` (other), `make_online_source.py`, `online/`, `lift2/`, `make_closure_base.py`, `verify_closure.py`, `check_log_table.py`, `inputs/factor_logs_*`, `inputs/*` (parent)	the parent's files, byte for byte (except `online/Cargo.toml`: one added `[[bin]]`)
`figures/profile.*`, `figures/power.*`, `make_figures.py`, `figures/design.*`	profile and power figures (SVG, PDF, PNG); design diagram (TikZ source, PDF, PNG)
`smoke/`	local correctness smoke of the frozen schedule
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`